42 SCHOOL · EMERGENCY PREP

Libft
3-Day Survival Guide

A brutal, no-fluff plan to pass your C evaluation — core concepts, memorizable code, and the exact questions peers will ask.

72	4	1
HOURS LEFT	STUDY MODULES	GOAL: PASS

- ✓ Must-know C concepts, isolated from fluff
- ✓ Code skeletons to memorize verbatim
- ✓ Exact peer / Moulinette defense questions
- ✓ Hour-by-hour 72h timeline

Generated from your Libft explanation file · Focus: C fundamentals for evaluation defense

1 The Absolute Must-Knows

Core C concepts you must genuinely understand — not just paste — to survive peer evaluation.

▸ Pointers & Memory

- `void *` carries no size info → must cast to `unsigned char *` / `char *` to index byte-by-byte.
- Data pointer (`content`) vs. the node holding it (`t_list`) — two separate allocations, two separate lifetimes.
- `t_list **lst` — why you need pointer-to-pointer to modify the caller's actual head variable, not a local copy. **This is the #1 question peers ask.**
- `struct s_list *next` inside the struct — why `t_list *next` can't be used there (the typedef isn't finished yet at that point).

▸ Memory Safety

- Every `malloc` must be checked (`if (ptr == NULL) return (NULL);`) before use.
- Overflow-safe multiplication (`ft_calloc`): rearrange `count * size` into `size > (size_t)-1 / count` so the check itself can't overflow.
- `memcpy` (no overlap handling, UB if overlapping) vs. `memmove` (direction-aware, checks `d > s` to copy backward/forward safely). **Single most-asked conceptual question.**
- `size_t` is unsigned — subtracting past 0 wraps to `SIZE_MAX` . Every bounded function (`strlcpy` , `strncmp` , `substr`) guards against this explicitly.
- `INT_MIN` negation overflow — solved in `ft_itoa` / `ft_putnbr_fd` by promoting to `long int` **before** negating.

▸ Header & Makefile Logic

- Include guards (`#ifndef / #define / #endif`) prevent double-declaration errors on multi-file includes.
- Makefile dependency logic: `%.o: %.c libft.h` — touching the header recompiles everything; touching one `.c` recompiles only that `.o` .
- "No unnecessary relinking" = running `make` twice produces zero output the second time.

> **DO THIS NOW**
> Test the double- `make` behavior yourself before your eval — don't assume it works.

▸ Function Pointers

- `char (*f)(unsigned int, char)` — declares `f` as a pointer to a function. Practice reading this out loud.
- `ft_lstmap` 's rollback: if node allocation fails mid-list, free the orphaned content **and** everything already built (`ft_lstclear`), so a failure never leaks partial data.

2 What to Memorize / Save Right Now

Skeletons the evaluator expects to see — save these verbatim.

▸ Classification Pattern (isalpha / isdigit / isascii / isprint — all identical shape)

```
int ft_isalpha(int c)
{
    if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z'))
        return (1);
    return (0);
}
```

▸ Byte-Iteration Pattern (memset / bzero)

```
ptr = (unsigned char *)b;
i = 0;
while (i < len)
{
    ptr[i] = (unsigned char)c;
    i++;
}
```

▸ memmove Direction Check — memorize this cold

```
if (d > s)
    // copy backward, high index first
else
    // copy forward, low index first
```

▸ Measure-Then-Allocate Pattern (strdup / substr / itoa — used everywhere)

1. measure length
2. `malloc(len + 1)`
3. check `NULL`
4. copy / build
5. return

▸ lstadd_front Rewiring — the exact 2-line order matters

```
new->next = *lst;   // MUST happen first
*lst = new;          // or you lose the old head
```

FT_SPLIT'S TWO STATIC HELPERS

Know *why* they're `static` (subject requires restricting scope, keeps functions under 25 lines) and the two-pass idea: count words first, then extract with a threaded index (`size_t *index`) that resumes scanning where the previous call stopped.

3 Common Defenses to Prepare

Be ready to answer these out loud, in your own words, pointing at your code.

Q — Why do both memmove and memcpy exist — walk me through an overlap case where memcpy breaks.

A — Draw it: src/dst overlapping, a forward copy overwrites bytes before they're read. memmove checks `d > s` and copies backward to avoid this.

Q — Why does ft_lstadd_front take t_list ** and not t_list *?

A — Local copy vs. address-of-the-caller's-variable — only a pointer-to-pointer lets you rewrite the caller's actual head.

Q — What happens if I pass NULL into this function?

A — It should not crash. Show your explicit NULL guards before any dereference.

Q — Why check dstsize == 0 before computing dstsize - 1 in strlcpy/strncmp?

A — size_t is unsigned — an unguarded subtraction wraps around to SIZE_MAX, turning a bounded loop into an unbounded one.

Q — Walk me through ft_itoa on INT_MIN.

A — Negating INT_MIN as a plain int overflows. The fix: promote to long int first, then negate — long has enough range to hold the positive equivalent safely.

Q — Show me make, then make again — why does nothing recompile the second time?

A — Be ready to demo this live. Per-file pattern rules mean only stale .o files rebuild; the archive step only re-runs if an object is newer than the .a.

Q — In ft_lstmap, what happens if the 5th of 10 allocations fails?

A — del() the orphaned content, ft_lstclear() the partial list built so far, return NULL — net heap footprint stays zero.

Q — Why is ft_split's helper function static?

A — Scope restriction to the translation unit — an explicit subject requirement, and it keeps every function under the 25-line limit.

Q — Norm questions

A — Two declarations then a blank line then executable code; single if + two explicit returns pattern; no ternaries anywhere.

KNOWN GAP — ADMIT IT IF ASKED

ft_split's partial-failure path (mid-array allocation failure) leaks earlier words. This is a known, documented limitation — don't get caught pretending otherwise. Own it confidently instead.

4 72-Hour Study Timeline

Brutal, hour-by-hour. Follow it exactly.

DAY 1 — Rebuild understanding, not memorization

HOURS	TASK
1–3	Recompile your project from scratch. Run `make`, `make`, `make bonus` — confirm zero unnecessary relinking. Fix immediately if broken.
4–6	Go function-by-function through 3.1–3.3 (char classification, memory primitives, string inspection). For each: write it on paper from memory, no copy-paste.
7–8	Test edge cases manually: NULL input, empty string, `len=0`, negative numbers into isdigit/atoi.
9–10	Break — but skim the Makefile section once more before bed.

DAY 2 — Heap allocation, string construction, linked lists (hardest material)

HOURS	TASK
1–3	ft_calloc, ft_strdup, ft_substr, ft_strjoin, ft_strtrim — rewrite from memory, focus on the measure-then-allocate pattern.
4–5	ft_split — trace through by hand with `"a,,b,c"` on paper. Understand count_words then alloc_word threading via `size_t *index`.
6–8	Linked list section — lstnew, lstadd_front/back, lstsize/lstlast, lstdelone/lstclear, lstiter, lstmap. Draw the pointer diagrams by hand for each.
9–10	Valgrind everything: `valgrind --leak-check=full` on a test main. Fix any leak now, not later.

DAY 3 — Defense prep + mock eval

HOURS	TASK
1–2	Re-read every "Technical Logic" explanation, out loud, as if explaining to a peer.
3–4	Have a friend (or solo, speaking aloud) grill you with the Section 3 defense questions.
5–6	Norm-check your code (norminette). Fix any violations — instant fail risk.
7–8	Full clean rebuild + Valgrind one more time. Confirm libft.a compiles with `-Wall -Wextra -Werror`, zero warnings.
9–10	Pre-eval buffer: sleep. Do not cram new material — review your memorized skeletons only.

Non-Negotiable Checkpoints Before You Walk In

- ☐ `make && make` = zero output on the second run
- ☐ Valgrind clean — no leaks, no invalid reads
- ☐ Norminette clean
- ☐ You can explain memmove's overlap direction from memory
- ☐ You can explain `t_list **` vs `t_list *` from memory